National University of Computer and Emerging Sciences

Lab Manual 03
Artificial Intelligence Lab

Department of Computer Science
FAST-NU, Lahore, Pakistan

1 Python Sets

Sets have following characteristics:

- Set in Python is a data structure equivalent to sets in mathematics.
- Sets are a mutable collection of distinct (unique) immutable values that are unordered.
- Any immutable data type can be an element of a set: a number, a string, a tuple.
- Mutable (changeable) data types cannot be elements of the set.
- In particular, list cannot be an element of a set (but tuple can), and another set cannot be an element of a set.
- You can perform standard operations on sets (union, intersection, difference).

1.1 Set Initialization Examples

You can initialize a set in the following ways:

```
# Initialize empty set
emptySet = set()

# Pass a list to set() to initialize it
dataScientist = set(['Python', 'R', 'SQL', 'Git', 'Tableau', 'SAS'])
dataEngineer = set(['Python', 'Java', 'Scala', 'Git', 'SQL', 'Hadoop'])

# Direct initialization using curly braces
dataScientist = {'Python', 'R', 'SQL', 'Git', 'Tableau', 'SAS'}
dataEngineer = {'Python', 'Java', 'Scala', 'Git', 'SQL', 'Hadoop'}

# Curly braces can only be used to initialize a set containing values
emptyDict= {} # type(emptyDict) is a dictionary
```

1.2 Set Modification Examples

Let's consider the following set for our add/remove examples:

```
# Initialize set with values
graphicDesigner = {'InDesign', 'Photoshop', 'Acrobat', 'Premiere', 'Bridge'}

# Add a new immutable element to the set
graphicDesigner.add('Illustrator')

# TypeError: unhasable type 'list'
graphicDesigner.add(['Powerpoint', 'Blender'])
# List cannot be added to a set

# Remove an element from the set
graphicDesigner.remove('Illustrator')

# Another way to remove an element. What is the difference?
graphicDesigner.discard('Premiere')
```

The discard() method removes the specified item from the set. This method is different from the remove() method, because the remove() method will raise an error if the specified item does not exist, and the discard() method will not.

```
# Remove and return an arbitrary value from a set, pop removes the first element from the set
graphicDesigner.pop()

# Remove all values from the set, set becomes empty
graphicDesigner.clear()
```

1.3 Set Operations

Python sets have methods that allow you to perform these mathematical operations like union, intersection, difference, and symmetric difference.
Let's initialize two sets to work on our examples:

```
# Initialize sets
dataScientist = set(['Python', 'R', 'SQL', 'Git', 'Tableau', 'SAS'])
dataEngineer = set(['Python', 'Java', 'Scala', 'Git', 'SQL', 'Hadoop'])

# set built-in function union
dataScientist.union(dataEngineer)

# Equivalent Result
dataScientist | dataEngineer

# Intersection operation
dataScientist.intersection(dataEngineer)

# Equivalent Result
dataScientist & dataEngineer

# These sets have elements in common so isdisjoint would return False
dataScientist.isdisjoint(dataEngineer)

# Difference Operation
dataScientist.difference(dataEngineer)

# Symmetric Difference Operation
dataScientist.symmetric_difference(dataEngineer)

# Equivalent Result
dataScientist ^ dataEngineer
```

1.4 Frozensets

You have encountered nested lists and tuples. The problem with nested sets is that you cannot normally have nested sets as sets cannot contain mutable values including sets.

- A frozenset is very similar to a set except that a frozenset is immutable.
- The primary reason to use them is to write clearer, functional code.
- By defining a variable as a frozen set, you're telling future readers: do not modify this.
- If you want to use a frozen set you'll have to use the function to construct it. No other way.

```
# Nested Lists and Tuples
nestedLists = [['the', 12], ['to', 11], ['of', 9], ['and', 7], ['that', 6]]
nestedTuples = (('the', 12), ('to', 11), ('of', 9), ('and', 7), ('that', 6))

# Initialize a frozenset
immutableSet = frozenset()

# Initialize a frozenset
nestedSets = set([frozenset()])
```

A major disadvantage of a frozenset is that since they are immutable, it means that you cannot add or remove values.

```
# AttributeError: 'frozenset' object has no attribute 'add'
immutableSet.add("Strasbourg")
```

2 Python Exception Handling

An exception is an error that is thrown by our code when the execution of the code results in an unexpected outcome. Normally, an exception will have an error type and an error message. Some examples are as follows.

```
ZeroDivisionError: division by zero
TypeError: must be str, not int
```

ZeroDivisionError and TypeError are the error type and the text that comes after the colon in the error message. The error message usually describes the error type.

2.1 Types of Exceptions

Here's a list of the common exceptions you'll come across in Python:

1. **ImportError**: It is raised when you try to import the library that is not installed or you have provided the wrong name
2. **IndexError:** Raised when an index is not found in a sequence. For example, if the length of the list is 10 and you are trying to access the 11th index from that list, then you will get this error

3. **IndentationError**: Raised when indentation is not specified properly
4. **ZeroDivisionError**: It is raised when you try to divide a number by zero
5. **ValueError**: Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified
6. **Exception**: Base class for all exceptions. If you are not sure about which exception may occur, you can use the base class. It will handle all of them

2.2 Exception Handling with Try Except Clause

Python provides us with the try except clause to handle exceptions that might be raised by our code. The basic anatomy of the try except clause is as follows:

```
try:
   // some code
except:
   // what to do when the code in try raise an exception
```

In plain English, the try except clause is basically saying, “Try to do this, except (otherwise) if there’s an error, then do this instead”.

There are a few options on what to do with the thrown exception from the `try` block. Let’s discuss them.

2.3 Re-raise the exception

Let’s take a look at how to write the try except statement to handle an exception by re-raising it. First, let’s define a function that takes two input arguments and returns their sum.

```
def myfunction(a, b):
   return a + b



myfunction(100, "one hundred")



print(" This WILL NOT be printed")
```

```
# this error will be given

   raiseTraceback (most recent call last):
  File "<input>", line 2, in <module>
  File "<input>", line 2, in myfunction
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Next, let's wrap it in a try except clause and pass input arguments with the wrong type

```
try:
    myfunction(100, "one hundred")
except:
    print("Error")
```

2.4 Catch certain types of exception

Another option is to define which exception types we want to catch specifically. To do this, we need to add the exception type to the `except` block.

```
try:
    myfunction(100, "one hundred")
except TypeError:
    print("Cannot sum the variables. Please pass numbers only.")


print("This WILL be printed")
```

To make it even better, we can actually log or print the exception itself.

```
try:
    myfunction(100, "one hundred")
except TypeError as e:
    print("Cannot sum the variables. The exception was:" ,e)

#Cannot sum the variables. The exception was: unsupported operand type(s) for +: 'int' and 'str'
```

Furthermore, we can catch multiple exception types in one `except` clause if we want to handle those exception types the same way. Let's pass an undefined variable to our function so that it will raise the `NameError`. We will also modify our `except` block to catch both `TypeError` and `NameError` and process either exception type the same way.

```
try:
    myfunction(100, a)
```

```
except (TypeError, NameError) as e:
    print("Cannot sum the variables. The exception was" , e)

#Cannot sum the variables. The exception was name 'a' is not defined
```

2.5 Try….Finally

So far the try statement had always been paired with except clauses. But there is another way to use it as well. The try statement can be followed by a **finally** clause. Finally clauses are called clean-up or termination clauses, because they must be executed under all circumstances, i.e. a "finally" clause is always executed regardless if an exception occurred in a try block or not. A simple example to demonstrate the finally clause:

```
try:
    x = float(input("Your number: "))
    inverse = 1.0 / x
finally:
    print("There may or may not have been an exception.")
print("The inverse: ", inverse)
```

```
Your number: 34
There may or may not have been an exception.
The inverse:  0.029411764705882353
```

2.6 Try..except and finally

"finally" and "except" can be used together for the same try block, as it can be seen in the following Python example:

```
try:
    x = float(input("Your number: "))
    inverse = 1.0 / x
except ValueError:
    print("You should have given either an int or a float")
except ZeroDivisionError:
    print("Infinity")
finally:
    print("There may or may not have been an exception.")
```

3. Python Object Oriented Programming

Python is a versatile programming language that supports various programming styles, including object-oriented programming (OOP) through the use of objects and classes.

An object is any entity that has attributes and behaviors. For example, a parrot is an object. It has

- attributes - name, age, color, etc.
- behavior - dancing, singing, etc.

Similarly, a class is a blueprint for that object.

Python Class and Object

```
class Parrot:

    # class attribute
    name = ""
    age = 0

# create parrot1 object
parrot1 = Parrot()
parrot1.name = "Blu"
parrot1.age = 10

# create another object parrot2
parrot2 = Parrot()
parrot2.name = "Woo"
parrot2.age = 15

# access attributes
print(f"{parrot1.name} is {parrot1.age} years old")
print(f"{parrot2.name} is {parrot2.age} years old")
```

Output

```
Blu is 10 years old
Woo is 15 years old
```

In the above example, we created a class with the name Parrot with two attributes: name and age.

Then, we create instances of the Parrot class. Here, parrot1 and parrot2 are references (value) to our new objects.

We then accessed and assigned different values to the instance attributes using the objects name and the . notation.

To learn more about classes and objects, visit Python Classes and Objects

---

Python Inheritance

Inheritance is a way of creating a new class for using details of an existing class without modifying it.

The newly formed class is a derived class (or child class). Similarly, the existing class is a base class (or parent class).

Example 2: Use of Inheritance in Python

```
# base class
class Animal:

    def eat(self):
        print( "I can eat!")

    def sleep(self):
        print("I can sleep!")

# derived class
class Dog(Animal):

    def bark(self):
        print("I can bark! Woof woof!!")

# Create object of the Dog class
dog1 = Dog()

# Calling members of the base class
dog1.eat()
dog1.sleep()
```

```
# Calling member of the derived class
dog1.bark();
```

Output

```
I can eat!
I can sleep!
I can bark! Woof woof!!
```

Here, dog1 (the object of derived class Dog) can access members of the base class Animal. It's because Dog is inherited from Animal.

```
# Calling members of the Animal class
dog1.eat()
dog1.sleep()
```

To learn more about inheritance, visit Python Inheritance.

Python Encapsulation

Encapsulation is one of the key features of object-oriented programming. Encapsulation refers to the bundling of attributes and methods inside a single class.

It prevents outer classes from accessing and changing attributes and methods of a class. This also helps to achieve data hiding.

In Python, we denote private attributes using underscore as the prefix i.e single _ or double __. For example,

```
class Computer:

    def __init__(self):
        self.__maxprice = 900

    def sell(self):
        print("Selling Price: {}".format(self.__maxprice))

    def setMaxPrice(self, price):
```

```
        self.__maxprice = price

c = Computer()
c.sell()

# change the price
c.__maxprice = 1000
c.sell()

# using setter function
c.setMaxPrice(1000)
c.sell()
```

Output

```
Selling Price: 900
Selling Price: 900
Selling Price: 1000
```

In the above program, we defined a Computer class.

We used __init__() method to store the maximum selling price of Computer. Here, notice the code

```
c.__maxprice = 1000
```

Here, we have tried to modify the value of __maxprice outside of the class. However, since __maxprice is a private variable, this modification is not seen on the output.

As shown, to change the value, we have to use a setter function i.e setMaxPrice() which takes price as a parameter.

Polymorphism

Polymorphism is another important concept of object-oriented programming. It simply means more than one form.

That is, the same entity (method or operator or object) can perform different operations in

different scenarios.

Let's see an example,

```
class Polygon:
    # method to render a shape
    def render(self):
        print("Rendering Polygon...")

class Square(Polygon):
    # renders Square
    def render(self):
        print("Rendering Square...")

class Circle(Polygon):
    # renders circle
    def render(self):
        print("Rendering Circle...")

# create an object of Square
s1 = Square()
s1.render()

# create an object of Circle
c1 = Circle()
c1.render()
```

Output

```
Rendering Square...
Rendering Circle...
```

In the above example, we have created a superclass: Polygon and two subclasses: Square and Circle. Notice the use of the render() method.

The main purpose of the render() method is to render the shape. However, the process of rendering a square is different from the process of rendering a circle.

Hence, the render() method behaves differently in different classes. Or, we can say render() is polymorphic.

Python Stacks

A stack is a data structure that stores items in a Last-In/First-Out manner. We will look at three different implementations of stacks in Python.

- Using list data structure
- Using collections.deque module
- Using queue.LifoQueue class

Stacks via Lists

The built-in Python list object can be used as a stack. For stack **.push()** method, we can use **.append()** method of list. **.pop()** method of list can remove elements in LIFO order. Popping an empty list (stack) will raise an **IndexError**. To get the top most item (peek) in the stack, write **list[-1]**. Bigger lists (stacks) often run into speed issues as they continue to grow. **list** may be familiar, but it should be avoided because it can potentially have memory reallocation issues.

```
myStack = []

myStack.append('a')
myStack.append('b')
myStack.append('c')

myStack        # ['a', 'b', 'c']

myStack.pop()                # 'c'
myStack.pop()                # 'b'
myStack.pop()                # 'a'
```

Stacks via collections.deque

This method solves the speed problem we face in lists. The **deque** class has been designed as such to provide O(1) time complexity for append and pop operations. The **deque** class is built on top of a doubly linked list structure which provides faster insertion and removal. Popping an empty **deque** gives the same **IndexError**. Read more about it here. Also, to know more about linked lists in Python, read this.

```
from collections import deque
myStack = deque()

myStack.append('a')
```

```
myStack.append('b')
myStack.append('c')
myStack        # deque(['a', 'b', 'c'])
myStack.pop()                    # ‘c’
myStack.pop()                    # ‘b’
myStack.pop()                    # ‘a’
```

Stacks via queue.LifoQueue

LifoQueue uses **.put()** and **.get()** to add and remove data from the stack. **LifoQueue** is designed to be fully thread-safe. But use it only if you are working with threads. Otherwise, **deque** works well. The **.get()** method by default will wait until an item is available. That means it waits forever if no item is present in the list. Instead, **get_nowait()** method would immediately raise empty stack error. Read more about it here.

```
from queue import LifoQueue
myStack = LifoQueue()

myStack.put('a')
myStack.put('b')
myStack.put('c')

myStack        # <queue.LifoQueue object at 0x7f408885e2b0>

myStack.get()          # ‘c’
myStack.get()          # ‘b’
myStack.get()          # ‘a’
```

Python Queues

A queue is FIFO data structure. The insert and delete operations are sometimes called **enqueue** and **dequeue**. We can use list as a queue as well. To follow FIFO, use **pop(0)** to remove the first element of the queue. But as discussed before, lists are slow. They are not ideal from a performance perspective.

We can use the **collections.deque** class again to implement Python queues. They work best for non-threaded programs. We can also use **queue.Queue** class. But it works well with synchronized programs.

If you are not looking for parallel processing, **collections.deque** is a good default choice.

```
from collections import deque
q = deque()
q.append('eat')
q.append('sleep')
```

```
q.append('code')
q                # deque(['eat', 'sleep', 'code'])
q.popleft()      # 'eat'
q.popleft()      # 'sleep'
q.popleft()      # 'code'
q.popleft()      # IndexError: "pop from an empty deque"
```

Questions:

1. Write a Python program to find the length of a set, apply all sets operations(union,intersection) and print the results,find maximum and the minimum value in a set, remove all elements from a given set and find the length of a set.
set1 = {1,2,3,4,5}
set2 = {4,5,6,7,8}

2. Write a program to count the number of vowels using sets in a given string.
sample output
Input : Hello World
Output : No. of vowels : 3

3. Write a program to add, mul, divide two numbers x and y. Implement exception handling techniques (try..except clause) for handling possible exceptions in the scenario.

4. Form a class queue and implement its functions using oop principles.

5. Form a queue using list data structure such that it works in LIFO order.

6. Create a class for rectangle shape that calculates its area based upon the length and width

7. Write a Python class named Circle constructed by a radius and two methods which will compute the area and the perimeter of a circle.